

Gaussian Blur

- Smooth an image by averaging nearby pixel values using Gaussian weights.
- **Core Idea:**
 - Each pixel's new value = weighted average of its neighborhood.
 - Weights come from the Gaussian function - higher near the center, lower farther away.
 - This ensures smooth transitions and natural-looking blur.

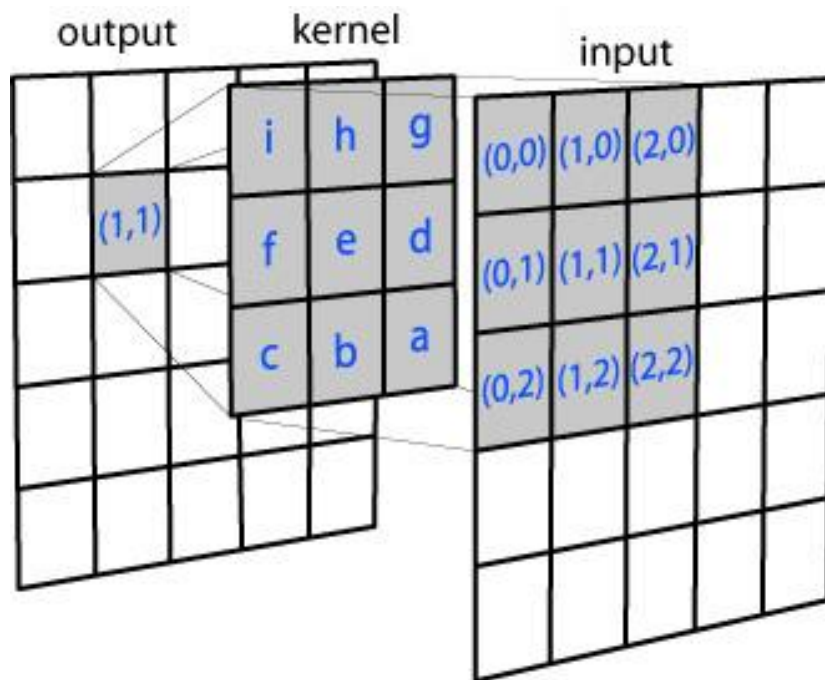
Algorithm

- Input: Image, kernel size (k), standard deviation (σ).
- Generate Gaussian kernel using σ .
- Pad image to handle border pixels
- For each pixel:
 - Multiply surrounding pixel values by kernel weights. $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$
 - Sum all results $\rightarrow$ assign as new pixel value.
- Output: Blurred image.

Key Points:

- Larger kernel or $\sigma \rightarrow$ stronger blur.
- Used before edge detection to reduce noise.

Illustration



How Kernel Values Are Calculated in Gaussian Blur

The Gaussian Function

Every pixel's weight in the kernel is computed using the 2D Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Where:

- x, y - distance from the center pixel (in kernel coordinates)
- σ - standard deviation (controls blur strength)
- $G(x, y)$ - weight (importance) for that pixel

Building the Kernel Grid

For a kernel of size $k \times k$:

- k is odd (3, 5, 7, ...) so there's a center pixel.
- Coordinates are taken from $-n$ to $+n$, where $n=(k-1)/2$

Example for a 3×3 kernel: $x, y \in \{-1, 0, 1\}$

Kernel calculation of each cell

For each (x, y), plug into the Gaussian equation.

Example ($\sigma = 1$):

Calculated $G(x,y)$ unnormalized:

$(-1,-1)$ 0.0585	$(-1,0)$ 0.965	$(-1,1)$ 0.0585
$(0,-1)$ 0.0965	$(0,0)$ 0.1592	$(0,1)$ 0.0965
$(1,-1)$ 0.0585	$(1,0)$ 0.0965	$(1,1)$ 0.0585

Calculated $G(x,y)$ unnormalized:

$$G(x, y) = \frac{G(x, y)}{\sum_{i,j} G(i, j)}$$

$(-1,-1)$ 0.0625	$(-1,0)$ 0.125	$(-1,1)$ 0.0625
$(0,-1)$ 0.125	$(0,0)$ 0.25	$(0,1)$ 0.125
$(1,-1)$ 0.0625	$(1,0)$ 0.125	$(1,1)$ 0.0625